PropOS uni-app 开发指南

> 文档版本：v1.0
>
> 日期：2026-04-09
>
> 适用范围：app/ 目录（uni-app 4.x + Vue 3 + TypeScript + Vite）
>
> 目标读者：参与 PropOS 移动端、小程序、Harmony 端开发的前端开发者

1. 文档定位

本指南以当前仓库的 app/ 实际代码为准，说明 PropOS uni-app 端的开发方式、目录职责、启动命令和新增功能流程。

请先明确两个前提：

1. 当前仓库的移动端实现仍然在 app/，不是 Flutter 工程。
2. docs/frontend/PAGE*SPEC*v1.7.md 中存在 Flutter 版页面描述，不能直接拿来指导当前 app/ 的编码实现。

换句话说，移动端当前应遵循的是“uni-app 代码现状 + .github/instructions/uniapp.instructions.md + 根级 copilot-instructions.md”的组合约束。

2. 当前项目现状

2.1 已落地部分

- 基础框架：uni-app 4.x、Vue 3、TypeScript、Vite
- 状态管理：Pinia setup store
- HTTP：luch-request，统一封装在 src/api/client.ts
- 日期展示：dayjs
- UI 库：wot-design-uni 已安装，但当前页面还未系统接入
- 鉴权基础链路已经打通：
  - App.vue 中通过 uni.addInterceptor 做全局导航拦截
  - src/api/client.ts 自动注入 Bearer Token
  - src/api/modules/auth.ts 提供登录、获取当前用户、登出、修改密码 API
  - src/stores/auth.ts 提供认证 store
  - src/pages/auth/login.vue 为当前唯一完整业务页面

2.2 当前页面状态

pages.json 已注册以下页面：

- 认证：登录页
- 总览：dashboard
- 资产：资产总览、楼栋详情、楼层热区图、房源详情
- 合同：合同列表、合同详情

- 财务：财务总览、账单、KPI
- 工单：工单列表、工单详情、新建工单
- 二房东：列表、详情

其中除登录页和认证链路外，其他页面目前大多还是占位骨架，后续开发应按模块逐步补齐 API、store 和组件。

2.3 当前明显缺口

- src/api/modules/ 目前只有 auth.ts，业务模块 API 尚未拆出
- src/stores/ 目前只有 auth.ts，业务 store 尚未建立
- 多数页面仍是“待实现”占位页
- pages.json 引用了 static/tabbar/ 下的 tabBar 图标，但当前仓库未看到对应资源，需要补齐
- app/ 目录当前没有提交 .env 文件，环境变量依赖本地注入

3. 技术栈与关键约束

3.1 技术栈

package.json 当前依赖如下：

- uni-app：@dcloudio/uni-app 3.0.0-4080420251103001
- Vue：3.4.x
- TypeScript：4.9.x
- Pinia：2.1.x
- luch-request：3.1.x
- dayjs：1.11.x
- wot-design-uni：1.6.x

3.2 必须遵守的架构规则

uni-app 端严格遵循单向数据流：

api -> store -> page/component

禁止出现以下情况：

- 在页面或组件中直接调用 uni.request、luch-request、fetch
- 在页面或组件中硬编码 /api/contracts 这类接口路径
- 在模板中堆叠复杂业务逻辑
- 在业务代码中直接写 20、30、90 这类魔法数字

3.3 当前仓库中的硬性编码规范

- API 路径统一定义在 src/constants/api_paths.ts
- 业务规则常量统一定义在 src/constants/business_rules.ts
- UI 常量统一定义在 src/constants/ui_constants.ts
- API 响应必须使用后端信封结构：
  - 成功：{ data, meta? }

    - 失败：{ error: { code, message } }
- Store 错误统一写入 error 字段，不透传原始异常对象
- 日期展示统一在组件层使用 dayjs(value).format('YYYY-MM-DD')
- 平台差异代码只能使用 uni-app 条件编译，不要用 userAgent 分支

4. 目录职责

当前 app/src 目录建议按下面的职责理解和扩展：

```
src/
  api/
    client.ts           # luch-request 统一封装，注入 token、错误包装、信封解析
    modules/            # 按业务域拆分 API 函数，当前只有 auth.ts
    index.ts            # API 桶导出
  constants/
    api_paths.ts        # 全部接口路径常量
    business_rules.ts   # 业务阈值常量
    ui_constants.ts     # 分页、布局、动画、轮询等 UI 常量
  pages/
    auth/
    dashboard/
    assets/
    contracts/
    finance/
    workorders/
    subleases/
  stores/
    auth.ts             # 当前已实现认证 store
    index.ts            # store 桶导出
  types/
    api.ts              # API 信封、分页元信息、ApiError 定义
  App.vue               # 全局导航守卫、全局颜色 token
  main.ts               # 创建 SSR App，挂载 Pinia
  pages.json            # 页面注册、tabBar、globalStyle
```

5. 本地开发与启动方式

5.1 前置条件

- 建议使用 Node.js 18+
- 先确保后端接口可访问
- 默认情况下，H5 开发代理会将 /api 转发到 http://localhost:8080

5.2 安装依赖

在 app/ 目录执行：

```
npm install
```

当前仓库未提交 lockfile，因此默认以 npm 命令为准；如果团队后续统一为 pnpm，需要连同 lockfile 一起落仓。

5.3 常用启动命令

```
# H5 本地调试
npm run dev:h5

# 微信小程序调试
npm run dev:mp-weixin

# Harmony 小程序调试
npm run dev:mp-harmony

# H5 生产构建
npm run build:h5

# 类型检查
npm run type-check
```

5.4 VITE_API_BASE_URL 的使用方式

当前代码中有两套后端地址策略：

1. H5 本地代理模式

- 不传 VITE*API*BASE_URL
- 直接启动 npm run dev:h5
- 前端访问 /api/**，由 vite.config.ts 代理到 http://localhost:8080

1. 直连后端模式

- 通过环境变量传入 VITE*API*BASE_URL
- 适合真机、跨设备联调、非本机后端地址

示例：

```
VITE_API_BASE_URL=http://192.168.1.20:8080 npm run dev:h5
VITE_API_BASE_URL=http://192.168.1.20:8080 npm run dev:mp-weixin
```

说明：

- src/api/client.ts 会将 VITE*API*BASE_URL 作为 baseURL
- vite.config.ts 也会读取该值作为 /api 代理目标
- 真机或小程序环境通常无法依赖本地 Vite 代理，因此更推荐显式传入可访问地址

6. 认证与导航机制

6.1 当前导航守卫实现

uni-app 端没有 vue-router，当前项目在 App.vue 中通过下面几类导航 API 的拦截实现鉴权：

- navigateTo
- redirectTo
- reLaunch
- switchTab

公开页面白名单目前只有：

- /pages/auth/login

如果未来新增免登录页面，必须同步更新 App.vue 中的 PUBLIC_PAGES。

6.2 登录态约定

- access*token* 和 *refresh*token 存在本地 storage
- 应用启动时，如果存在 access_token，会自动调用 authStore.fetchMe()
- 登录成功后跳转 /pages/dashboard/index
- 401 时 API 客户端会尝试刷新 token，一次失败后回到登录页

6.3 当前 Token 刷新行为说明

src/api/client.ts 中的刷新逻辑不是“自动透明重放原请求”，而是：

1. 401 时尝试调用刷新接口
2. 刷新成功后写回新 token
3. 抛出 ApiError('TOKEN_REFRESHED', '已刷新 Token，请重试', 401)

因此，涉及幂等操作或重试体验的页面，需要开发者自行决定是否在上层做一次重试。不要误以为底层已经自动把原请求重放了。

7. API 层开发规范

7.1 统一通过 client.ts 出口调用

可用方法：

- apiGet
- apiGetList
- apiPost
- apiPatch
- apiDelete

新增业务 API 时，统一放到 src/api/modules/<domain>.ts，再由 src/api/index.ts 桶导出。

7.2 API 模块示例

```
import { apiGetList, apiGet } from '@/api/client'
import { API_BUILDINGS } from '@/constants/api_paths'
import type { PaginationMeta } from '@/types/api'

export interface BuildingSummary {
  id: string
  name: string
  propertyType: 'office' | 'retail' | 'apartment'
```

```
  occupancyRate: number
}

export interface BuildingListParams {
  page: number
  pageSize: number
}

export const assetsApi = {
  listBuildings: (params: BuildingListParams) =>
    apiGetList<BuildingSummary>(API_BUILDINGS, params),
  getBuilding: (id: string) =>
    apiGet<BuildingSummary>(`${API_BUILDINGS}/${id}`),
}
```

规则：

- URL 前缀统一来自 api_paths.ts
- 返回值直接使用信封解包后的强类型结果
- 不在 API 模块里处理页面状态
- 不在 API 模块里写 toast、跳转等 UI 行为

8. Store 层开发规范

8.1 统一使用 Pinia setup store

业务 store 统一建议保留以下状态字段：

- list
- item
- loading
- error
- meta

认证 store 可以保留 profile 这类领域化字段，但新增业务 store 不建议偏离这一模式。

8.2 Store 模板

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import { ApiError } from '@/types/api'
import { assetsApi } from '@/api/modules/assets'
import type { BuildingSummary } from '@/api/modules/assets'
import type { PaginationMeta } from '@/types/api'
import { DEFAULT_PAGE_SIZE } from '@/constants/ui_constants'

export const useAssetsStore = defineStore('assets', () => {
  const list = ref<BuildingSummary[]>([])
  const item = ref<BuildingSummary | null>(null)
  const loading = ref(false)
  const error = ref<string | null>(null)
  const meta = ref<PaginationMeta | null>(null)
```

```
  async function fetchList(page = 1) {
    loading.value = true
    error.value = null
    try {
      const res = await assetsApi.listBuildings({
        page,
        pageSize: DEFAULT_PAGE_SIZE,
      })
      list.value = res.data
      meta.value = res.meta
    } catch (e) {
      error.value = e instanceof ApiError ? e.message : '操作失败，请重试'
    } finally {
      loading.value = false
    }
  }

  return {
    list,
    item,
    loading,
    error,
    meta,
    fetchList,
  }
})
```

规则：

- 页面不直接调 API，只调 store action
- catch 中统一转换错误文案
- 不把原始异常对象挂到响应式状态
- 如果 store 代码超过 200 行，按子领域拆分

9. 页面与组件开发规范

9.1 页面职责

页面只做三件事：

1. 触发 store action
2. 读取 store state
3. 组合展示组件

页面里不要直接做：

- HTTP 请求
- 接口路径拼接
- 大段数据转换逻辑
- 业务日期计算

9.2 页面最小模式

```
<script setup lang="ts">
import { onMounted } from 'vue'
import { useAssetsStore } from '@/stores/assets'

const assetsStore = useAssetsStore()

onMounted(() => {
  void assetsStore.fetchList()
})
</script>

<template>
  <view>
    <view v-if="assetsStore.loading">加载中...</view>
    <view v-else-if="assetsStore.error">{{ assetsStore.error }}</view>
    <view v-else>
      <view v-for="item in assetsStore.list" :key="item.id">
        {{ item.name }}
      </view>
    </view>
  </view>
</template>
```

9.3 页面注册要求

新增页面后必须同步修改 src/pages.json，否则 uni-app 不会识别该页面。

如果页面是 tabBar 页面，还要同时处理：

- tabBar.list 配置
- 对应图标资源
- 登录态导航逻辑

9.4 复杂度限制

- 单个页面建议控制在 250 行以内
- template 嵌套超过 4 层时，应提取子组件
- 多处复用逻辑提到 composables/
- 通用展示块提到 components/

10. UI、样式与跨端约定

10.1 颜色 token

当前全局颜色变量定义在 App.vue：

- --color-success
- --color-warning
- --color-danger

- --color-neutral
- --color-primary

状态语义固定如下：

- leased / paid -> success
- expiring_soon / warning -> warning
- vacant / overdue / error -> danger
- non_leasable -> neutral

不要在业务页面内直接写绿色、红色、橙色字面量来表达状态。

10.2 尺寸与展示常量

统一优先复用 ui_constants.ts 中的常量，例如：

- DEFAULT*PAGE*SIZE
- MAX*PAGE*SIZE
- CONTENT*MAX*WIDTH_PX
- CARD*MAX*WIDTH_PX
- ANIM*DURATION*MS
- ALERT*POLL*INTERVAL_MS

10.3 日期规则

- API 传输时间使用 ISO 8601 字符串
- 展示层用 dayjs 格式化
- WALE、逾期天数、NOI 相关业务计算都放在后端，不在前端重复计算

10.4 条件编译

平台差异必须使用 uni-app 条件编译：

```
<!-- #ifdef MP-WEIXIN -->
<view>仅微信小程序显示</view>
<!-- #endif -->

<!-- #ifdef APP-HARMONY -->
<view>仅 Harmony 显示</view>
<!-- #endif -->
```

不要使用 navigator.userAgent 或 process.env 自己分支平台逻辑。

10.5 关于 wot-design-uni

wot-design-uni 已经安装，但当前仓库仍主要使用原生 uni 组件。建议策略如下：

- 新增复杂表单、列表、弹层时，优先评估使用 wot-design-uni
- 同一页面内避免无规则混搭太多原生组件和 Wot 组件
- 即使使用 Wot，也仍然遵循当前项目的颜色 token、错误处理和 store 分层规则

11. 推荐开发流程

新增一个业务模块时，建议按下面顺序推进：

1. 在 src/types/ 明确接口结构或复用现有类型
2. 在 src/constants/api_paths.ts 增加接口路径常量
3. 在 src/api/modules/<domain>.ts 增加 API 方法
4. 在 src/stores/<domain>.ts 建立 Pinia setup store
5. 在页面中接入 store，不直接请求接口
6. 在 src/stores/index.ts 和 src/api/index.ts 做桶导出
7. 如为新页面，注册到 src/pages.json
8. 补充静态资源、空态、错误态、加载态
9. 执行 npm run type-check
10. 至少在 H5 和目标平台各验证一次

12. 模块落地优先级建议

基于当前仓库状态，建议优先顺序如下：

1. 补齐 assets、contracts、finance、workorders、subleases 的 API modules
2. 为上述模块建立对应 store
3. 将占位页面替换为真实列表页
4. 统一抽出列表空态、错误态、分页和状态标签组件
5. 最后再补充跨端差异优化与体验增强

原因很简单：当前最大短板不是 UI，而是业务链路没有从“页面骨架”接到“API + store + 页面”的完整闭环。

13. 提交前自检清单

每次提交前，至少检查以下内容：

- 页面里是否出现了直接 HTTP 请求
- 是否硬编码了 API 路径或业务常量
- 是否把新页面写进了 pages.json
- 是否遵循了错误处理统一模式
- 是否使用了 dayjs 而不是直接操作 Date
- 是否误把业务计算放到了前端
- 是否补齐了目标平台所需静态资源
- 是否执行了 npm run type-check

14. 当前项目的建议共识

最后给当前仓库一个务实结论：

- app/ 现在更像“已完成基础设施、尚未完成业务模块”的工程骨架

- 后续开发不要再写第二套前端模式，继续围绕现有 uni-app 架构补模块即可
- 如果未来确实要切换 Flutter，应先完成架构决策并停止 app/ 的增量开发；在此之前，当前移动端研发都应以本指南为准